C# / .NET Learning Roadmap

Topic Notes — Section 11: Version Control (Git)

Topics: 11.1 through 11.4

Date: 29 June 2026

Section 11 — Version Control (Git)

This section covers all four roadmap topics: 11.1 (Init, clone, commit, push, pull), 11.2 (Branching and merging), 11.3 (Resolving merge conflicts), 11.4 (Pull requests, commit conventions).

Note on context: Git is a tool, not a language feature. The focus is on the workflows and commands you will use daily as a .NET developer on a team.

11.1. Init, Clone, Commit, Push, Pull

Builds on: Nothing -- Git is standalone. This is the foundation of all other Git topics.

What is Git and why does it exist?

Git is a distributed version control system. It tracks every change to every file, who made it, when, and why. Without version control teams end up with files named OrderService_v2_FINAL2.cs. Git solves three problems: history (every change recorded), collaboration (multiple people work simultaneously without overwriting each other), and safety (any version can be restored).

The three areas of Git

Understanding these three areas is the key to understanding every Git command:

```
Working Directory    Staging Area (Index)    Repository (.git)
■■■■■■■■■■■■■■■■■■■■    ■■■■■■■■■■■■■■■■■■■■■■■■    ■■■■■■■■■■■■■■■■■■■■
Your actual files    Changes queued          Committed snapshots
You edit here        for next commit         (permanent history)

     git add  ■■■■■■■■■■■■■■■■>
                               git commit ■■■■■■■■■■■>
     git checkout <■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

git init and git clone

```
# Start tracking a new project:
mkdir MyProject && cd MyProject
git init
# Creates .git folder -- Git now tracks everything here

# Copy an existing repository:
git clone https://github.com/org/my-dotnet-app.git

# Clone into a specific folder name:
git clone https://github.com/org/my-dotnet-app.git MyApp

# After cloning -- full history, all branches, all files:
cd my-dotnet-app
git log --oneline -5
```

git add -- stage changes

```
# Stage one specific file:
```

```
git add src/OrderService.cs

# Stage all changes in a folder:
git add src/

# Stage everything changed or new:
git add .

# Stage parts of a file interactively (hunk by hunk):
git add -p src/OrderService.cs

# See what is staged vs what is not:
git status
git diff           # unstaged changes
git diff --staged  # staged changes (going into next commit)
```

git commit -- save a snapshot

```
# Commit with a message:
git commit -m "feat: add order cancellation validation"

# Stage tracked files and commit in one step:
# (new/untracked files still need git add first)
git commit -am "fix: null check in OrderValidator"

# Amend the last commit (before pushing):
git commit --amend -m "fix: null check -- handle edge case"

# See commit history:
git log
git log --oneline          # compact view
git log --oneline --graph  # with branch graph
git log --oneline -10      # last 10 commits
```

git push and git pull

```
# Push commits to the remote:
git push origin main

# Push a new branch for the first time:
git push -u origin feature/order-cancellation
# -u sets upstream -- after this, just 'git push' works

# Force push safely (fails if someone else pushed first):
git push --force-with-lease origin feature/my-branch

# Pull (fetch + merge) from remote:
git pull origin main

# Pull with rebase instead of merge (cleaner history):
git pull --rebase origin main

# Fetch without merging (look before you leap):
git fetch origin
git log origin/main  # see what is there
```

```
git merge origin/main  # merge when ready
```

git status and git log -- your daily companions

```
# What is going on right now:
git status
# On branch feature/order-cancellation
# Changes to be committed:
#   modified: src/Services/OrderService.cs
# Changes not staged for commit:
#   modified: src/Models/Order.cs
# Untracked files:
#   src/Services/CancellationService.cs

# Who changed a file and when:
git log --oneline src/Services/OrderService.cs

# What changed in a specific commit:
git show abc1234
```

The .gitignore file -- what not to track

```
# .gitignore at project root:

# Build output -- always exclude:
bin/
obj/

# IDE files:
*.user
.vs/
.idea/
.DS_Store

# Secrets -- NEVER commit these:
appsettings.Development.json
*.pfx
secrets.json

# NuGet packages (restored automatically):
packages/
*.nupkg
```

Key Takeaways for 11.1

- Git tracks history, enables collaboration, and provides safety
- Three areas: Working Directory -> Staging Area -> Repository
- git add stages; git commit saves a snapshot; git push sends to server
- git pull brings remote changes; git clone copies a remote repo
- .gitignore prevents bin/, obj/, secrets from being tracked

```
Command               | What it does
----------------------+-------------------------------
git init              | Start tracking a new project
```

```
git clone <url>       | Copy an existing repository
git status            | See what has changed
git add <file>        | Stage a change
git commit -m "msg"   | Save a snapshot
git push origin main  | Send commits to remote
git pull origin main  | Get remote commits
git log --oneline     | See commit history
```

Memory aid: Git is a time machine for your code. git add loads the time capsule. git commit seals it. git push sends it to a vault. git pull retrieves someone else's capsule from the vault.

Debugging Tips

Symptom	Cause	Fix
git push rejected -- non-fast-forward	Remote has commits you do not have locally	git pull --rebase origin main, then push again
Accidentally staged wrong file	git add . staged unintended files	git restore --staged to unstage without losing changes
Committed secret key or large file	Sensitive file now in Git history	Use BFG Repo Cleaner to remove; rotate the secret immediately
git status shows nothing but files clearly changed	.gitignore may be ignoring the file, or wrong directory	Check .gitignore rules; ensure you are in the repo root

11.2. Branching and Merging

Builds on: 11.1 (commits -- branches are pointers to commits), knowing how to add/commit/push.

What are branches and why do they exist?

A branch is a lightweight, movable pointer to a commit. Without branches, everyone commits directly to main -- you cannot work on a feature without affecting everyone else. Branches let you isolate work until it is ready.

Real-world analogy: Branches are parallel universes. main is the stable production universe. You create a new universe (feature/order-cancellation), make changes there, and only merge back when complete and tested.

git branch -- create, list, delete

```
# List all branches (* = current):
git branch
# * main
#   feature/order-cancellation

# Create a new branch:
```

```
git branch feature/order-export

# Create and switch in one command (modern):
git switch -c feature/order-export
# Older syntax (same result):
git checkout -b feature/order-export

# Delete a merged branch:
git branch -d feature/order-export

# Force delete an unmerged branch:
git branch -D feature/abandoned-idea

# List remote branches:
git branch -r

# List all (local + remote):
git branch -a
```

The standard .NET team branching workflow

```
main      <- production-ready, always deployable
develop   <- integration branch (some teams skip)
feature/* <- one branch per feature or task
bugfix/*  <- bug fixes
hotfix/*  <- urgent production fixes
release/* <- release preparation

# Typical daily workflow:

# 1. Start from latest main:
git switch main
git pull origin main

# 2. Create a feature branch:
git switch -c feature/JIRA-123-order-cancellation

# 3. Work on feature -- commit often:
git add src/Services/OrderService.cs
git commit -m "feat: add order cancellation validation"

# 4. Push to remote so others can see your work:
git push -u origin feature/JIRA-123-order-cancellation

# 5. Open a Pull Request (covered in 11.4)
```

git merge -- combining branches

```
# Merge feature into main:
git switch main
git merge feature/order-cancellation

# Fast-forward -- linear history (main has not moved):
# main:    A■■B■■C■■D■■E
# (feature commits added to main directly)
```

```
# Merge commit -- preserves branch history:
git merge --no-ff feature/order-cancellation
# main:    A■■B■■C■■■■■■M
#                \    /
# feature:        D■■E
# --no-ff forces merge commit even if fast-forward possible
```

git rebase -- cleaner linear history

```
# Rebase feature branch on top of latest main:
git switch feature/order-cancellation
git rebase main

# Before rebase:
# main:    A■■B■■C
# feature: A■■B■■D■■E  (D, E are your commits)

# After rebase:
# main:    A■■B■■C
# feature: A■■B■■C■■D'■■E'  (replayed on top of C)

# Result: clean linear history

# Merge vs rebase:
# Merge:  + preserves true history
#         - creates merge commits
# Rebase: + clean linear history
#         - rewrites commit hashes
#         - NEVER rebase a shared branch
```

Stashing -- saving work without committing

```
# Save work-in-progress without committing:
git stash
# Working directory is now clean

# Switch branches, do other work, come back:
git switch hotfix/urgent-bug
# ... fix the bug ...
git switch feature/order-cancellation

# Restore stashed work:
git stash pop    # applies and removes the stash
git stash apply  # applies but keeps the stash

# See all stashes:
git stash list
# stash@{0}: WIP on feature: partial order export

# Stash with a descriptive message:
git stash save "half-done order export feature"
```

Key Takeaways for 11.2

- Branches isolate work -- develop independently until ready to merge

- git switch -c branch-name creates and switches in one command
- Fast-forward merge = linear history; --no-ff = explicit merge commit
- Rebase replays your commits on another branch -- clean but rewrites hashes
- NEVER rebase a branch that others are working on
- git stash saves uncommitted work so you can switch branches safely

Memory aid: A branch is a sticky note on a commit. It moves forward as you commit. When you merge, you move the sticky note on main to point to the merge result.

Debugging Tips

Symptom	Cause	Fix
Switched branches but old files still showing	Uncommitted changes -- Git kept them in working dir	git status to check; git stash if you want a clean switch
Rebase stopped with conflicts	Conflict during replay -- same as merge conflict	Resolve file, git add, git rebase --continue
Lost work after git checkout -f	-f discards uncommitted changes permanently	Always git stash or git status before -f; use git switch instead
Branch not showing up after push	Pushed to wrong remote name or typo in branch name	git branch -a to verify; git push -u origin correct-name

11.3. Resolving Merge Conflicts

Builds on: 11.2 (branching and merging -- conflicts happen during merges), 11.1 (git status -- used to find conflicting files).

What is a merge conflict and why does it happen?

A conflict happens when two branches change the SAME lines of the same file. Git can merge automatically when changes are on different lines. When they overlap, Git cannot decide whose version is correct -- it asks you to decide.

```
main branch changed line 15:
    public decimal CalcTax(decimal amount) => amount * 0.18m;

feature branch changed line 15:
    public decimal CalcTax(decimal amount) => amount * 0.20m;

Git cannot merge automatically -- which rate is correct?
```

What a conflict looks like in a file

```
public class TaxCalculator
{
<<<<<<< HEAD
    // Current branch (main):
    public decimal CalcTax(decimal amount) => amount * 0.18m;
=======
    // Incoming branch (feature/update-tax-rates):
    public decimal CalcTax(decimal amount) => amount * 0.20m;
```

```
>>>>>>> feature/update-tax-rates

    public string GetSummary(decimal amount)
        => $"Tax: {CalcTax(amount):C}";
}

# Markers:
# <<<<<<< HEAD       -- your current branch version starts
# =======            -- divider
# >>>>>>> branch     -- incoming branch version ends
```

Resolving the conflict -- step by step

```
# Step 1: Attempt the merge:
git merge feature/update-tax-rates
# CONFLICT (content): Merge conflict in TaxCalculator.cs

# Step 2: Find conflicting files:
git status
# Both modified: TaxCalculator.cs

# Step 3: Open the file and decide:

# Option A -- keep your version (HEAD):
public decimal CalcTax(decimal amount) => amount * 0.18m;

# Option B -- keep their version:
public decimal CalcTax(decimal amount) => amount * 0.20m;

# Option C -- combine both (most common):
public decimal CalcTax(decimal amount, string region)
    => region == "UK" ? amount * 0.20m : amount * 0.18m;

# Step 4: Remove ALL conflict markers
# File must compile cleanly before proceeding

# Step 5: Stage the resolved file:
git add TaxCalculator.cs

# Step 6: Complete the merge:
git commit -m "Merge feature/update-tax-rates"
#   -- resolve tax rate conflict
```

Using VS Code / Visual Studio merge tool

Most editors have built-in conflict resolution UI. In VS Code, open the conflicting file -- it shows Accept Current Change, Accept Incoming Change, Accept Both Changes buttons above each conflict block. Click the option that matches your decision, edit the result if needed, save, then git add and git commit.

```
# Open Git's configured merge tool:
git mergetool

# VS Code shows three panels:
# LEFT:   your version (HEAD/Current)
```

```
# RIGHT:  incoming version (the branch)
# CENTER: the result you are building
```

Preventing conflicts -- the best strategy

```
# Pull main frequently into your feature branch:
git switch feature/order-cancellation
git pull origin main  # bring latest main into your branch

# Or with rebase:
git rebase origin/main

# The earlier you integrate, the smaller the conflicts

# Abort a merge or rebase if things go wrong:
git merge --abort
git rebase --abort
```

Key Takeaways for 11.3

- Conflicts happen when two branches change the same lines of the same file
- Git marks conflicts with <<<<<<< , =======, >>>>>>> -- you resolve by editing
- Steps: edit file -> remove all markers -> git add -> git commit
- VS Code / Visual Studio have visual merge tools -- use them
- Best prevention: short-lived branches, pull main frequently, communicate
- git merge --abort or git rebase --abort to start over if needed

Memory aid: A merge conflict is two people trying to sit in the same plane seat. Git stops and says 'you figure out who sits where.' You decide, tell Git, and the flight continues.

Debugging Tips

Symptom	Cause	Fix
Merge commit created but code still shows conflict markers	Committed without removing all <<<<<<< markers	Search the file for <<<<<<< ; resolve and amend the commit
git merge --abort fails	Already committed the conflict resolution	Use git revert to undo the merge commit instead
Conflict in generated file (migrations, .csproj)	Two branches modified the same auto-generated file	Accept one version, regenerate the file correctly, commit
Same conflict appears repeatedly	Same lines keep diverging on long-lived branches	Use git rerere to record resolutions; merge main more frequently

11.4. Pull Requests and Commit Conventions

Builds on: 11.2 (branching -- PRs are how branches get reviewed before merging), 11.1 (commits -- conventions apply to every commit message).

What is a Pull Request (PR)?

A Pull Request (Merge Request in GitLab) is a formal request to merge your branch into another branch, with an opportunity for teammates to review the code before it is merged. PRs are a platform feature (GitHub, Azure DevOps, GitLab) -- not a Git feature. They are the standard workflow at virtually every professional .NET team.

```
Developer workflow:
1. Create feature branch from main
2. Write code and commit
3. Push branch to remote
4. Open Pull Request:
   'Please review and merge feature/X into main'
5. Teammates review -- comment, request changes, approve
6. PR merged into main (squash, merge commit, or rebase)
7. Feature branch deleted
```

Opening a PR on GitHub

```
# 1. Push your branch:
git push -u origin feature/JIRA-123-order-cancellation

# 2. GitHub shows a prompt:
# 'feature/JIRA-123-order-cancellation recently pushed.
#  Compare & pull request.' -- click it.

# 3. Fill in the PR:
# Title:     JIRA-123: Add order cancellation with validation
# Reviewers: @teammate1, @teammate2
# Labels:    feature, needs-review
# Linked:    #123 (closes the issue when merged)
```

A good PR description template

```
## What does this PR do?
Adds order cancellation with validation rules:
- Shipped orders cannot be cancelled
- Cancellation records timestamp and user
- Sends cancellation confirmation email

## Why?
JIRA-123: Support team needs to cancel orders
directly from the admin portal.

## How to test?
1. Create an order and note the ID
2. Call DELETE /api/orders/{id} with auth token
3. Verify status changes to 'Cancelled'
4. Verify confirmation email sent

## Checklist
- [x] Unit tests added
- [x] Integration tests added
- [x] No new compiler warnings
- [x] Documentation updated
```

Commit conventions -- Conventional Commits

```
# BAD -- tells you nothing:
git commit -m "fix"
git commit -m "changes"
git commit -m "wip"
git commit -m "update OrderService.cs"

# GOOD -- tells you what, why, and scope:
git commit -m "feat: add order cancellation validation"
git commit -m "fix: prevent cancelling shipped orders"
git commit -m "test: add unit tests for CancelAsync"
git commit -m "refactor: extract validation to OrderValidator"
git commit -m "docs: document cancellation API endpoint"
git commit -m "chore: upgrade EF Core from 8.0 to 9.0"
```

The Conventional Commits standard format:

```
<type>(<scope>): <description>

[optional body]

[optional footer]

# Types:
feat      New feature for the user
fix       Bug fix for the user
test      Adding or updating tests
refactor  Code change -- no bug fix, no new feature
docs      Documentation only
chore     Build process, tooling, dependencies
ci        CI/CD configuration
perf      Performance improvement
style     Formatting only -- no logic change
revert    Reverts a previous commit
```

Commit examples with scope

```
git commit -m \
  "feat(orders): add order cancellation endpoint"

git commit -m \
  "fix(validator): handle null CustomerName in dto"

git commit -m \
  "test(OrderService): add cancellation edge cases"

git commit -m \
  "refactor(pricing): extract discount strategies"

# Breaking change -- ! after type:
git commit -m "feat!: change order ID from int to GUID"
```

Commit message best practices

```
# Subject line rules:
```

```
# - 50 characters or fewer
# - Imperative mood: 'Add' not 'Added' or 'Adding'
# - No period at the end
# - Capitalise the first word after the colon

# Good:  feat: add order validation
# Bad:   feat: added order validation.
# Bad:   feat: Adding order validation

# Body -- when you need more context:
git commit -m "fix(orders): prevent double-cancel race

When two requests cancel the same order simultaneously,
the second cancellation would succeed incorrectly.
Added optimistic concurrency check using row version.

Closes #456"
```

Squash commits before merging

```
# Typical feature branch history before cleanup:
# abc1234 wip
# def5678 fix tests
# ghi9012 more wip
# jkl3456 actually fix the tests
# mno7890 done

# Squash into one clean commit:
git rebase -i HEAD~5  # interactive rebase last 5 commits

# Editor opens -- change 'pick' to 'squash' (or 's'):
# pick abc1234 wip
# squash def5678 fix tests
# squash ghi9012 more wip
# squash jkl3456 actually fix the tests
# squash mno7890 done

# Result: one clean commit:
# abc9999 feat: add order cancellation with validation

# Most teams use 'Squash and merge' on the PR platform
# so you do not need to do this manually
```

Branch protection rules

```
# Typical rules for main branch:

Require pull request before merging         (on)
Require at least 1 approval                 (on)
Dismiss stale reviews on new commits        (on)
Require status checks to pass (CI)          (on)
Require branches to be up to date           (on)
Do not allow force pushes                   (on)
Do not allow deletions                      (on)
```

```
# These rules ensure:
# - No direct commits to main
# - Every change is reviewed
# - CI passes before merge
```

Key Takeaways for 11.4

- PRs are how branches get code-reviewed before merging -- platform feature
- Good PR: clear title, what/why/how-to-test description, checklist
- Conventional Commits: type(scope): description
- Types: feat, fix, test, refactor, docs, chore, ci, perf
- Commit messages: imperative mood, 50 char subject, explain why not what
- Squash WIP commits before merging -- keep history clean and readable
- Branch protection on main: require PR, require review, require CI pass

```
Type      | Use for
----------+------------------------------
feat      | New feature
fix       | Bug fix
test      | Tests added or updated
refactor  | Code restructured
docs      | Documentation
chore     | Build, tooling, dependencies
ci        | CI/CD changes
perf      | Performance improvement
```

Memory aid: A PR is a job interview for your code. Your feature branch applies for a position on main. Reviewers are the hiring committee. They can approve, request changes, or reject. Branch protection rules are HR policies that cannot be bypassed.

Debugging Tips

Symptom	Cause	Fix
PR shows merge conflicts in the UI	main has moved since you branched	git pull origin main (or rebase) locally, resolve, push again
CI fails on PR but passes locally	Missing env variable, different .NET version, or test order dependency	Check CI logs; add missing env vars; fix test isolation
Reviewer cannot see latest commits in PR	Pushed to wrong branch or forgot to push	git push origin feature/your-branch to update the PR
Squash rebase went wrong -- commits lost	Rebase dropped commits or merge conflict during rebase	git reflog to find the lost commits; cherry-pick them back

---

Section 11 -- Complete Key Takeaways

```
Topic           | Core concept              | Key commands
----------------+---------------------------+-------------------
11.1 Init/Clone | Three areas: dir->stage-> | add, commit,
                | repo                      | push, pull
11.2 Branching  | Isolate work in parallel  | switch -c, merge,
                | universes                 | rebase, stash
```

```
11.3 Conflicts  | Same lines on two branches| Edit, add, commit
11.4 PRs+Conv.  | Review gate + history     | Conventional
                |                           | Commits, squash
```

.NET-specific Git rules:

- Always add bin/, obj/, .vs/ to .gitignore -- never commit build output
- Never commit appsettings.Development.json with real secrets
- One logical change per commit -- do not mix features with reformatting
- Branch prefixes: feature/, bugfix/, hotfix/ for clarity
- PR title should reference ticket: JIRA-123: Add order cancellation

---

Section 11 complete. All 4 roadmap topics (11.1 through 11.4) covered: Init/Clone/Commit/Push/Pull, Branching and Merging, Resolving Merge Conflicts, Pull Requests and Commit Conventions. Say next to begin Section 12: Modern .NET -- Application Hosting and Configuration.